

LAPORAN AKHIR PRAKTIKUM

Mata Praktikum : Algoritma Pemrograman 2
Kelas : IIA04
Praktikum ke- : 1
Tanggal : 07 April 2026
Materi : Thread dan Multithread
NPM : 50425612
Nama : Maulana Abdul Aziz
Ketua Asisten : Ni Gusti Ayu
Jumlah Lembar : 5



LABORATORIUM TEKNIK INFORMATIKA
UNIVERSITAS GUNADARMA

2026

A. Single Thread (Eksekusi Berurutan)

```
import time

def olah_data(data):
    print(f"Memproses {data}...")
    time.sleep(1) # simulasi waktu pemrosesan yang diizinkan per thread (1 proses thread = 1 detik)

if __name__ == "__main__":
    start_time = time.perf_counter()
    data_list = [f"Data-{i}" for i in range(1, 11)] # simulasi total thread yang prosesnya dieksekusi secara bergantian/berurut

    for r in data_list:
        olah_data(r)

    print(f"Total Waktu: {time.perf_counter() - start_time:.2f} detik")

... Memproses Data-1...
Memproses Data-2...
Memproses Data-3...
Memproses Data-4...
Memproses Data-5...
Memproses Data-6...
Memproses Data-7...
Memproses Data-8...
Memproses Data-9...
Memproses Data-10...
Total Waktu: 10.00 detik
```

Kode ini mensimulasikan pemrosesan 10 data secara berurutan tanpa menggunakan thread tambahan. Setiap item data diproses satu per satu, artinya proses berikutnya baru bisa dimulai setelah proses sebelumnya selesai sepenuhnya.

Alur kerja:

- **import time** : Mengimpor modul bawaan Python untuk mengukur waktu eksekusi program dan mensimulasikan durasi proses menggunakan `time.sleep()`.
- **def olah_data(data)** : Fungsi yang mencetak nama data yang sedang diproses, kemudian memanggil `time.sleep(1)` untuk mensimulasikan bahwa setiap proses membutuhkan waktu 1 detik.
- **data_list = [f"Data-{i}" for i in range(1, 11)]** : Membuat daftar 10 item data (Data-1 hingga Data-10) menggunakan list comprehension.
- **for r in data_list: olah_data(r)** : Memproses setiap item secara berurutan. Karena tidak ada yang berjalan paralel, total waktu menjadi $10 \times 1 \text{ detik} = 10 \text{ detik}$.
- **time.perf_counter()** : Digunakan untuk mengukur waktu eksekusi dengan presisi tinggi, lebih akurat dibandingkan `time.time()`.

Hasil output memperlihatkan setiap data diproses dari Data-1 hingga Data-10 secara berurutan, dengan total waktu 10.00 detik sesuai ekspektasi untuk model eksekusi sequential.

B. Multithread (Eksekusi Bersamaan)

```
import threading # import built-in function yang digunakan untuk simulasi multithreading
import time

def olah_data(data):
    print(f"Memproses {data}...")
    time.sleep(1) # simulasi waktu pemrosesan yang diizinkan untuk melakukan seluruh proses (seluruh proses thread = 1 detik)

if __name__ == "__main__":
    start_time = time.perf_counter()
    threads = []
    data_list = [f"Data-{i}" for i in range(1, 11)] # simulasi total thread yang prosesnya dieksekusi secara bersamaan dalam satu waktu

    for r in data_list:
        t = threading.Thread(target=olah_data, args=(r,))
        threads.append(t)
        t.start()

    for t in threads:
        t.join()

    print(f"Total Waktu: {time.perf_counter() - start_time:.2f} detik")

Memproses Data-1...
Memproses Data-2...
Memproses Data-3...
Memproses Data-4...
Memproses Data-5...
Memproses Data-6...
Memproses Data-7...
Memproses Data-8...
Memproses Data-9...
Memproses Data-10...
Total Waktu: 1.00 detik
```

Kode ini memproses 10 data yang sama seperti pada kode sebelumnya, namun setiap item data dijalankan dalam thread terpisah sehingga semua proses dapat berjalan secara bersamaan (concurrent).

- **import threading** : Mengimpor modul bawaan Python untuk membuat dan mengelola thread.
- **threads = []** : List kosong untuk menyimpan referensi semua thread yang dibuat, agar dapat di-join

setelahnya.

- **threading.Thread(target=olah_data, args=(r,))** : Membuat objek thread baru. Parameter target menentukan fungsi yang akan dieksekusi thread, dan args berisi argumen yang diteruskan ke fungsi tersebut.
- **t.start()** : Memulai eksekusi thread. Setelah dipanggil, thread langsung berjalan secara concurrent tanpa menunggu thread lain selesai terlebih dahulu.
- **t.join()** : Memerintahkan program utama untuk menunggu hingga thread tersebut selesai sebelum melanjutkan ke instruksi berikutnya. Tanpa join(), program utama bisa mencetak total waktu sebelum semua thread benar-benar selesai.

Karena semua 10 thread dijalankan hampir bersamaan dan masing-masing menunggu 1 detik secara paralel (bukan bergantian), total waktu eksekusi menjadi hanya 1.00 detik, yakni 10 kali lebih cepat dibandingkan single thread. Ini menunjukkan keunggulan multithread untuk tugas-tugas yang bersifat I/O-bound.

C. Multithreading dengan threading.Lock()

```
import threading
import time

class DataProcessor:
    def __init__(self):
        # Membuat lock untuk sinkronisasi
        self.lock = threading.Lock()

    def process_data(self, thread_name):
        # Menggunakan 'with self.lock' secara otomatis akan melakukan acquire() dan release()
        with self.lock:
            print(f'Thread {thread_name} sedang memproses data...')
            # Simulasi proses data
            time.sleep(1)

def run_thread(data_processor, name):
    for i in range(5):
        data_processor.process_data(name)

if __name__ == "__main__":
    # Membuat satu objek processor yang akan dipakai bersama oleh semua thread
    processor = DataProcessor()

    # Membuat thread
    thread1 = threading.Thread(target=run_thread, args=(processor, "A"))
    thread2 = threading.Thread(target=run_thread, args=(processor, "B"))

    # Memulai thread
    thread1.start()
    thread2.start()

    # Menunggu thread selesai
    thread1.join()
    thread2.join()

    print("Semua proses selesai.")
```

```
... Thread A sedang memproses data...
Thread B sedang memproses data...
Thread A sedang memproses data...
Thread B sedang memproses data...
Thread B sedang memproses data...
Thread A sedang memproses data...
Thread A sedang memproses data...
Thread A sedang memproses data...
Thread B sedang memproses data...
Thread B sedang memproses data...
Semua proses selesai.
```

Kode ini memperkenalkan skenario di mana dua thread berbagi satu sumber daya yang sama, yaitu objek DataProcessor. Tanpa mekanisme pengunci, kedua thread dapat mengakses sumber daya tersebut secara bersamaan dan menyebabkan race condition. Solusinya adalah menggunakan threading.Lock().

- **class DataProcessor** : Kelas yang merepresentasikan pemroses data bersama. Satu objek dari kelas ini digunakan secara bersama oleh semua thread (shared resource).
- **self.lock = threading.Lock()** : Membuat objek Lock sebagai atribut kelas. Lock ini berperan sebagai "kunci pintu", di mana hanya satu thread yang bisa memegang kunci pada satu waktu.
- **with self.lock** : Context manager yang secara otomatis memanggil lock.acquire() saat memasuki blok dan lock.release() saat keluar, bahkan jika terjadi exception sekalipun. Cara ini jauh lebih aman daripada memanggil acquire/release secara manual.
- **def run_thread(data_processor, name)** : Fungsi yang dijalankan oleh setiap thread. Masing-masing thread memanggil process_data() sebanyak 5 kali.

- **thread1** dan **thread2** : Dua thread dibuat dengan nama A dan B, keduanya menggunakan objek processor yang sama sebagai shared resource.

Hasil output memperlihatkan Thread A dan Thread B bergantian mengakses sumber daya secara teratur. Lock memastikan hanya satu thread yang berada di dalam blok kritis pada satu waktu, sehingga data terlindungi dari korupsi dan race condition tidak terjadi.

Essay - Perbedaan Thread dan Multithread

Definisi

Sebelum membahas perbedaannya, perlu dipahami terlebih dahulu apa yang dimaksud dengan masing-masing istilah. Thread adalah unit eksekusi terkecil yang dimiliki oleh sebuah proses. Setiap program yang berjalan pasti memiliki minimal satu thread yang disebut main thread atau thread utama. Thread berbagi ruang memori (heap) yang sama dalam satu proses, namun memiliki stack eksekusi, program counter, dan register sendiri-sendiri.

Istilah "multithread" merujuk pada teknik pemrograman di mana sebuah program menggunakan lebih dari satu thread secara bersamaan dalam satu proses. Jika single thread berarti program berjalan dengan hanya satu jalur eksekusi, multithread berarti program memiliki banyak jalur eksekusi yang berjalan secara concurrent.

Perbedaan Utama

Perbedaan mendasar antara keduanya terletak pada model eksekusinya:

Aspek	Single Thread	Multithread
Model eksekusi	Berurutan (sequential)	Bersamaan (concurrent)
Waktu proses	Akumulasi semua tugas	Mendekati tugas terlama
Kompleksitas kode	Sederhana	Lebih kompleks
Risiko race condition	Tidak ada	Ada, perlu sinkronisasi
Penggunaan CPU/core	Satu core saja	Dapat memanfaatkan multi-core
Cocok untuk	Tugas berurutan/bergantung	Tugas I/O-bound yang independen
Debugging	Mudah dan deterministik	Lebih sulit, non-deterministik

Analisis

Dalam konteks praktikum ini, perbedaan paling nyata terlihat dari hasil pengukuran waktu. Single thread memproses 10 data secara berurutan dengan total waktu 10.00 detik, sedangkan multithread menyelesaikan tugas yang sama hanya dalam 1.00 detik. Penghematan sebesar 90% ini terjadi karena pada multithread, semua thread menjalankan `time.sleep(1)` secara bersamaan, bukan bergantian.

Namun, keunggulan multithread tidak bersifat universal. Python memiliki mekanisme Global Interpreter Lock (GIL) yang membatasi hanya satu thread yang dapat mengeksekusi bytecode Python pada satu waktu. Artinya, multithread di Python tidak cocok untuk tugas CPU-bound seperti komputasi matematika berat, karena GIL mencegah thread berjalan sungguh-sungguh paralel. GIL baru dilepaskan ketika thread sedang melakukan operasi I/O seperti `time.sleep()`, pembacaan file, atau request jaringan. Itulah mengapa skenario pada praktikum ini tetap menunjukkan percepatan yang signifikan meskipun menggunakan Python.

Selain masalah performa, multithread juga membawa tantangan baru berupa race condition. Race condition terjadi ketika dua atau lebih thread mencoba membaca dan menulis data yang sama secara bersamaan, sehingga menghasilkan output yang tidak dapat diprediksi dan tidak konsisten. Untuk menangani ini, digunakan mekanisme sinkronisasi seperti Lock (Mutex), Semaphore, atau Event. Dalam praktikum ini, Lock digunakan untuk memastikan hanya satu thread yang dapat masuk ke blok kode kritis pada satu waktu.

Kesimpulan

Single thread lebih tepat digunakan ketika tugas-tugas memiliki ketergantungan satu sama lain, ketika urutan

eksekusi sangat penting, atau ketika kompleksitas kode perlu dijaga seminimal mungkin. Multithread menjadi pilihan unggul ketika program perlu menjalankan banyak tugas I/O-bound secara bersamaan, seperti melayani banyak permintaan web server, membaca banyak file sekaligus, atau memproses data dalam jumlah besar yang bersifat independen satu sama lain.

Pemahaman terhadap perbedaan ini sangat krusial bagi seorang programmer, karena pemilihan paradigma eksekusi yang tepat akan berdampak langsung pada efisiensi, keandalan, dan kemudahan pemeliharaan sebuah aplikasi.